

MODULE 1 — CLINICAL NLP PREPROCESSING Type: Custom build | Effort: Medium | Publishable: No

WHAT IT DOES Takes raw doctor input — typed text, Hinglish, abbreviations — and transforms it into clean structured dental language for downstream modules.

INPUT 'pt c/o severe pain UL6 since 3d, worsening on cold'

OUTPUT 'Patient has severe pain in upper left molar for 3 days, worsening with cold stimulation'

WHAT IT DOES (DETAILED)

- Dental abbreviation expansion — UL6 → upper left first molar, RCT → root canal treatment, OHI → oral hygiene instructions
- Language detection and transliteration — handles Hindi, Hinglish, and code-mixed input
- Sentence segmentation — splits multi-complaint inputs into discrete communicable units
- Named entity recognition — tags ANATOMY, SYMPTOM, PROCEDURE, INSTRUCTION, SEVERITY, DURATION separately

HOW TO BUILD IT

- spaCy base model fine-tuned on dental clinical notes using i2b2 clinical NLP dataset as bootstrap
- Custom NER labels: ANATOMY, SYMPTOM, PROCEDURE, INSTRUCTION, SEVERITY, DURATION
- Abbreviation dictionary: ~200 common dental shorthand terms, curated with ISL expert
- IndicNLP library for Hindi/Hinglish tokenisation before spaCy processing

INTERNAL ARCHITECTURE Input: raw text → Language detect (IndicNLP) → Transliterate if Hindi/Hinglish → Abbreviation expansion (dict lookup) → spaCy NER tagging → Sentence segmentation Output: structured JSON {entities, clean_text, segments}

TOOLS spaCy v3, IndicNLP, i2b2 clinical NLP corpus, custom dental abbreviation dictionary, Python

KEY RISK If NER misclassifies an anatomy term as a symptom, ISL grammar ordering in M4 will be wrong. Evaluate NER F1 per label class — do not report aggregate accuracy only.

MODULE 2 — LLM DECOMPOSITION Type: API (not custom) | Effort: Low | Publishable: No

WHAT IT DOES Takes Tier 3 dental terms — procedures with no established ISL sign — and decomposes them into sequences of concrete visual concepts mappable to existing ISL primitives.

INPUT 'root canal treatment' (flagged as Tier 3 — no ISL sign exists)

OUTPUT ['tooth', 'inside', 'infected', 'clean', 'seal']

WHAT IT DOES (DETAILED)

- Three-chained LLM calls: (1) clinical simplification, (2) visual concept extraction, (3) ISL primitive suggestion
- Validation layer: checks each output concept against primitive vocabulary JSON before passing downstream
- Confidence scoring: if cosine similarity to nearest primitive < 0.75 , flag for fingerspelling fallback
- Dental-domain prompt engineering — constrained to ISL primitive vocabulary, max 6 output concepts per term

HOW TO BUILD IT

- Call 1 system prompt: "Explain this dental term using only concrete visible actions and objects. Max 12 words. No jargon. As if explaining to a non-literate adult."
- Call 2 system prompt: "From this description extract only things a person can see, point to, or mime. JSON array of 3–6 single concepts only."
- Call 3 system prompt: "For each concept map to the closest item in this vocabulary list: [your 150 primitives]. Return JSON with concept and matched_primitive."
- Custom validation: Python post-processor checks all matched_primitives exist in sign vocabulary JSON

INTERNAL ARCHITECTURE Input: Tier-3 dental term (string) → Call 1 → plain description → Call 2 → concept list (JSON array) → Call 3 → primitive mapping (JSON) → Validation layer (cosine similarity check) → Fallback: flag low-confidence → fingerspell Output: ordered primitive ID list OR fingerspell flag

TOOLS Claude Sonnet API, Python requests, BioSentVec for validation, JSON schema validation, custom prompt templates

KEY RISK LLM outputs are non-deterministic — same input can give different concept lists on different runs. Cache outputs per term and have your ISL expert validate the cache before production use.

MODULE 3 — ISL PRIMITIVE MAPPER Type: Custom build | Effort: High | Publishable: Yes — core contribution

WHAT IT DOES Maps any plain-language concept to a specific sign ID in the ISL primitive vocabulary. The mapping function and the dental vocabulary it maps to are both novel.

INPUT ['tooth', 'inside', 'infected', 'clean', 'seal']

OUTPUT [SIGN_042, SIGN_118, SIGN_077, SIGN_031, SIGN_095]

WHAT IT DOES (DETAILED)

- Semantic similarity model: embeds input concept and all 150 primitives, returns nearest match above threshold
- Fine-tuned on dental primitive vocabulary using BioSentVec medical domain embeddings
- Handles synonyms: 'aching' → same sign as 'pain', 'molar' → same sign as 'back tooth'
- Three-tier output: high confidence → direct sign ID, medium → top-3 for expert review, low → fingerspell fallback

HOW TO BUILD IT

- Build vocabulary JSON: {id, term, description, isl_video_url, mediapipe_pose_path} for all 150 primitives
- Fine-tune BioSentVec: create (concept, primitive) positive pairs from expert validation sessions
- Threshold calibration: plot precision-recall curve on held-out 20% of vocabulary, pick threshold at F1 peak
- Evaluation set: 100 dental concepts manually mapped to primitives by ISL expert — measure auto-mapper accuracy against this gold standard

INTERNAL ARCHITECTURE Input: concept list (strings) → Embed each concept (BioSentVec) → Embed all 150 primitives (cached) → Cosine similarity matrix → Threshold filter (0.75) → High confidence → SIGN_ID → Medium → top-3 for review → Low → fingerspell flag Output: [SIGN_ID, SIGN_ID, FINGERSPELL, ...]

TOOLS BioSentVec, HuggingFace sentence-transformers, scikit-learn cosine similarity, Python numpy, custom vocabulary JSON

KEY RISK Generic sentence transformers will fail on dental terms not in their training data. BioSentVec is non-negotiable — it was trained on PubMed abstracts and has medical concept coverage.

MODULE 4 — ISL GRAMMAR REORDERING Type: Custom build | Effort: Medium | Publishable: Yes — core contribution

WHAT IT DOES Reorders the sign sequence from M3 into grammatically correct ISL word order. ISL has topic-comment structure, spatial verb agreement, and non-manual markers — this is not signed English.

INPUT [SIGN_tooth, SIGN_inside, SIGN_infected, SIGN_clean, SIGN_seal] — English concept order

OUTPUT [SIGN_tooth, SIGN_inside, SIGN_infected, SIGN_tooth_TOPIC, SIGN_clean, SIGN_seal] — ISL topic-comment order + NMM flags

WHAT IT DOES (DETAILED)

- Rule-based reordering engine: 7 grammar rules for dental encounter contexts, validated by ISL expert
- Topic-comment ordering: anatomy/body part always first (topic), then description/action (comment)
- Dental-specific rules: pain location before pain descriptor, procedure action before result, instruction verb before object
- Non-manual marker insertion: flags positions where headshake (negation), raised eyebrows (yes/no question), or forward lean (emphasis) should occur — passed as metadata to M5

HOW TO BUILD IT

- Implement as Python rule engine: each rule is a pattern matcher on NER entity types + sign semantic categories
- Rule 1: ANATOMY before SYMPTOM (tooth before pain, not pain before tooth)
- Rule 2: SEVERITY modifier follows the sign it modifies (pain SEVERE, not SEVERE pain)
- Rule 3: Instructions: VERB → OBJECT → TIME (rinse mouth 30 seconds)
- Validate every rule with ISL expert: show 20 before/after examples, confirm naturalness rating $\geq 4/5$

INTERNAL ARCHITECTURE Input: [SIGN_ID list] + NER entity types from M1 → Identify entity roles (ANATOMY, SYMPTOM, ACTION) → Apply Rule 1: anatomy-first ordering → Apply Rule 2: severity placement → Apply Rule 3–7: dental-specific patterns → Insert NMM metadata flags Output: reordered [SIGN_ID list] + NMM flags

TOOLS Python rule engine (custom), spaCy dependency parser, ISL grammar reference (Vasishtha et al.), expert validation scoring sheet

KEY RISK Grammar rules derived without Deaf community input will feel unnatural even if technically correct. Every rule must be validated by your ISL expert before production. Budget 2–3 sessions for rule iteration.

MODULE 5 — SIGN RENDERER Type: Custom build | Effort: High | Publishable: Yes — supporting contribution

WHAT IT DOES Converts the ordered sign ID sequence into visible ISL output. Built in two stages: video-stitching prototype for quick validation, upgraded to pose-based skeletal animation for the research version.

INPUT [SIGN_042, SIGN_118, SIGN_077, SIGN_031, SIGN_095] + NMM flags

OUTPUT Smooth ISL avatar animation in browser (Three.js skeleton) or stitched MP4

WHAT IT DOES (DETAILED)

- Stage A prototype: ffmpeg-based video stitching — concatenate sign videos with transition blending

- Stage B research: MediaPipe Holistic pose extraction → skeletal pose sequences as JSON → Three.js avatar renderer
- NMM rendering: facial expression overlays triggered at flagged frame positions from M4
- Fingerspelling renderer: separate letter-by-letter animation pathway for low-confidence terms

HOW TO BUILD IT

- Stage A: Python moviepy + ffmpeg, 0.3s hold at end of each sign, 0.15s linear blend, 25fps output
- Stage B pose extraction: MediaPipe Holistic on each sign video → extract 33 pose + 21L + 21R hand landmarks per frame → save as JSON array
- Stage B interpolation: linear lerp between last frame of sign N and first frame of sign N+1 over 8 transition frames
- Stage B rendering: Three.js SkeletonHelper with 17 bone hierarchy, landmark positions mapped to bone rotations via inverse kinematics
- Fingerspelling: pre-extracted ISL alphabet pose sequences, concatenated at 0.8s per letter

INTERNAL ARCHITECTURE Input: [SIGN_ID list] + NMM flags → Lookup pose JSON per SIGN_ID (cached) → Concatenate pose sequences → Interpolate transitions (8 frames) → Inject NMM overlays at flagged positions → Three.js skeleton renderer (browser) OR ffmpeg stitch (Stage A fallback) Output: live avatar animation in browser

TOOLS MediaPipe Holistic (Python), Three.js + SkeletonHelper, ffmpeg + moviepy (Stage A), Python numpy for interpolation, WebSocket for real-time streaming

KEY RISK Pose interpolation between signs with very different hand positions looks physically unnatural. Identify jump cuts in your vocabulary and add manual keyframe fixes for the most common transitions.